

Introduction

Low-Level Design (LLD) interviews evaluate your ability to transform real-world scenarios into clean, scalable, and easy-to-maintain code structures. One of the most commonly discussed examples is the **Vending Machine System**.

In this article, we'll guide you through a clear 9-step framework to design this system, just like you would in a real interview. Along the way, we'll share pro tips to help you articulate your reasoning, handle edge cases, and impress your interviewer with a well-thought-out approach.

Step 1: Clarify Requirements

Functional Requirements:

- The system should support multiple products with different prices and quantities.
- The system should accept notes of different denominations (\$1, \$5, \$10, \$20, \$50, \$100).
- The system should dispense selected products and return change if necessary.
- The system should track available products and quantities (inventory management).
- The system should handle multiple transactions concurrently and ensure data consistency.
- The system should provide admin interface for restocking products and collecting money.
- The system should handle exceptional scenarios (insufficient funds, out-of-stock products).
- The system should ensure operations are only allowed in appropriate states.

- The system should manage the following operational states:
 - **IDLE:** Ready to accept new transactions, users can browse and select products.
 - **PROCESSING_PAYMENT:** User has selected product and is paying, can insert money or cancel.
 - **DISPENSING:** Payment complete, machine is dispensing product and change.
 - **OUT_OF_SERVICE:** Machine is not operational due to maintenance or malfunction.

Interview Tip: Always ask the interviewer to confirm the requirements before jumping into code. It shows clarity and alignment.

Edge Cases to Consider

- Concurrent access to same product.
 - Insufficient change in machine.
 - Power failure.
 - Product out of stock during transaction.
 - Payment failure scenarios.
-

Step 2: Identify Core Entities

1. VendingMachine (Core Entity)

- id: int [PK]
- location: String
- currentState: VendingMachineState
- isOperational: boolean

2. Product

- id: int [PK]
- name: String
- price: Double
- category: ProductCategory (Enum: BEVERAGE, SNACK, CANDY, etc.)

3. Inventory

- productId: int [FK]
- vendingMachineId: int [FK]
- quantity: int
- minThreshold: int (for restocking alerts)

4. CashBox

- id: int [PK]
- vendingMachineId: int [FK]
- denominations: Map<Denomination, Integer> (denomination → count)
- totalAmount: Double

5. Denomination (Enum)

- ONE_DOLLAR(100)
- FIVE_DOLLAR(500)
- TEN_DOLLAR(1000)
- TWENTY_DOLLAR(2000)
- FIFTY_DOLLAR(5000)
- HUNDRED_DOLLAR(10000)

6. Transaction

- id: int [PK]
- vendingMachineId: int [FK]
- productId: int [FK]
- amountInserted: Double
- amountRequired: Double

- changeReturned: Double
- status: TransactionStatus (Enum: PENDING, COMPLETED, FAILED, CANCELLED)
- timestamp: long

7. TransactionStatus (Enum)

- PENDING
- COMPLETED
- FAILED
- CANCELLED

8. ProductCategory (Enum)

- BEVERAGE
- SNACK
- CANDY
- CHIPS
- COOKIES
- OTHER

9. Recovery (Power Failure Recovery)

- id: int [PK]
- vendingMachineId: int [FK]
- transactionId: int [FK]
- state: VendingMachineState (PROCESSING_PAYMENT or DISPENSING)
- status: RecoveryStatus (Enum: PENDING, COMPLETED)
- createdAt: long
- completedAt: long (nullable)

10. RecoveryStatus (Enum)

- PENDING
- COMPLETED

Interview Tip: Clearly identifying entities and enums at this stage helps define relationships (like Product ↔ Inventory, VendingMachine ↔ CashBox) and ensures smooth implementation of State and Recovery mechanisms in the system.

Step 3: Visualize Interaction Flows

1. Product Listing & Inventory Flow:

- GET /api/products →
 - System fetches all products →
 - System checks inventory levels →
 - System returns product list with availability status (in stock/out of stock) →
 - Frontend displays products with real-time stock information
-

2. Payment Processing Flow:

- POST /api/payment →
- User selects products →
- System calculates total amount →
- System validates inventory →
- System processes payment →
- System updates inventory →
- System returns success with transaction details

Error Scenarios: Inventory check → Payment validation → Stock update → Transaction logging

3. Payment Cancel/Failure Flow:

- **POST /api/payment/cancel** OR Payment fails →
- System rolls back inventory changes →
- System returns inserted money →
- System logs failure reason →
- System resets transaction state →
- System returns to IDLE state

Error Scenarios: Insufficient funds, out of stock, user cancellation, system failure

4. Admin Stocking Flow:

- **POST /api/admin/stock** (Admin only) →
- System validates admin credentials →
- System updates product quantities →
- System resets out-of-stock alerts →
- System logs restocking activity →
- System returns updated inventory status

Admin Operations: Add products, update quantities, view sales reports, collect cash

5. Power Failure Recovery Flow:

- System startup →
- `RecoveryController.checkAndRecover()` →
- `RecoveryService.performRecovery()` →
- System checks pending recoveries →
- System executes recovery based on interrupted state →
- System returns to IDLE state

Recovery Strategies: `PROCESSING_PAYMENT` = refund money, `DISPENSING` = complete dispensing + change

Step 4: Defines Class Structures & Relationships

Layers of Architecture

We will design the **architecture layers** of the system in a structured way, ensuring **separation of concerns** and **modularity**. The system will be organized into the following layers:

Client/UI Layer → Controller Layer → Service Layer → Domain Layer

Each layer has a distinct role:

- **Client/UI Layer:** Provides the interface for end users or external systems to interact with the vending machine (e.g., touchscreen displays, mobile payment apps, APIs).
- **Controller Layer:** Acts as the entry point for user requests, orchestrates calls to services, and converts input/output between

the UI and business logic (e.g., selecting a product, inserting money, initiating a transaction).

- **Service Layer:** Contains the business logic, rules, and workflows for handling product selection, inventory management, cash handling, transaction processing, dispensing, and recovery mechanisms.
 - **Domain Layer:** Defines the core entities, states, and relationships that model the real-world vending machine system (e.g., VendingMachine, Product, Inventory, Transaction, CashBox, Recovery), independent of infrastructure or UI.
-

CONTROLLERS:

1. VendingMachineController (Product & Inventory Management)

- - List<Product> getAvailableProducts(int machineId)
- - Product getProductDetails(int machineId, int productId)
- - InventoryStatus getInventoryStatus(int machineId)

2. PaymentController (Payment Processing)

- - Transaction processPayment(int machineId, PaymentRequest request)
- - void cancelPayment(int machineId, int transactionId)
- - PaymentStatus getPaymentStatus(int machineId, int transactionId)
- - List<Transaction> getTransactionHistory(int machineId)

3. RecoveryController (Power Failure Recovery)

- - void checkAndRecover(int machineId)
- - RecoveryStatus getRecoveryStatus(int machineId)
- - void markRecoveryComplete(int machineId, int recoveryId)

4. AdminController (Admin Operations)

- - void restockProduct(int machineId, RestockRequest request)
 - - void collectCash(int machineId)
 - - SalesReport getSalesReport(int machineId, DateRange dateRange)
-

SERVICES:

1. VendingMachineService (Product & Inventory Management)

- - List<Product> getAvailableProducts(int machineId)
- - Product getProductDetails(int machineId, int productId)
- - InventoryStatus getInventoryStatus(int machineId)
- - void updateInventory(int machineId, int productId, int quantity)

2. PaymentService (Payment Processing)

- - Transaction processPayment(int machineId, PaymentRequest request)
- - void cancelPayment(int machineId, int transactionId)
- - PaymentStatus getPaymentStatus(int machineId, int transactionId)
- - void updateCashBox(int machineId, Map<Denomination, Integer> denominations)

3. RecoveryService (Power Failure Recovery)

- - void performRecovery(int machineId)
- - RecoveryStatus getRecoveryStatus(int machineId)
- - void createRecoveryEntry(int machineId, int transactionId, VendingMachineState state)
- - void markRecoveryComplete(int machineId, int recoveryId)

4. AdminService (Admin Operations)

- - void restockProduct(int machineId, RestockRequest request)

- - void collectCash(int machineId)
 - - SalesReport getSalesReport(int machineId, DateRange dateRange)
-

REPOSITORIES:

1. VendingMachineRepository

- - VendingMachine findById(int machineId)
- - void updateInventory(int machineId, int productId, int quantity)

2. PaymentRepository

- - void saveTransaction(Transaction transaction)
- - List<Transaction> findByMachine(int machineId)
- - Transaction findById(int transactionId)
- - void updateCashBox(int machineId, Map<Denomination, Integer> denominations)

3. ProductRepository

- - List<Product> findByMachine(int machineId)
- - Product findById(int productId)

4. RecoveryRepository

- - void saveRecovery(Recovery recovery)
 - - List<Recovery> findPendingRecoveries(int machineId)
 - - Recovery findById(int recoveryId)
 - - void markComplete(int recoveryId)
-

STATE PATTERN IMPLEMENTATION:

1. VendingMachineState (Interface)

- + void processPayment(VendingMachine machine, PaymentRequest request)
- + void cancelPayment(VendingMachine machine, int transactionId)
- + String getStateName()

2. IdleState (Concrete State)

- + void processPayment(VendingMachine machine, PaymentRequest request) *// Valid transition → moves to PROCESSING_PAYMENT*
- + void cancelPayment(VendingMachine machine, int transactionId) *// No state change*
- + String getStateName() *// Returns "IDLE"*

3. ProcessingPaymentState (Concrete State)

- + void processPayment(VendingMachine machine, PaymentRequest request) *// Invalid (already processing)*
- + void cancelPayment(VendingMachine machine, int transactionId) *// Valid - transitions back to IDLE*
- + String getStateName() *// Returns "PROCESSING_PAYMENT"*

4. DispensingState (Concrete State)

- + void processPayment(VendingMachine machine, PaymentRequest request) *// Invalid*
- + void cancelPayment(VendingMachine machine, int transactionId) *// Invalid (dispensing in progress)*
- + String getStateName() *// Returns "DISPENSING"*

5. VendingMachine (Context - Uses State Pattern)

- - id: int
- - location: String
- - currentState: VendingMachineState
- - currentTransaction: Transaction

- - inventory: Map<Product, Integer>
 - - cashBox: CashBox
 - - isOperational: boolean
 - + void setState(VendingMachineState newState)
 - + Transaction processPayment(PaymentRequest request)
 - + void cancelPayment(int transactionId)
 - + String getCurrentStateName()
-

STEP-5: CORE USE CASES & METHODS

1. Product Listing Use Case:

- GET /api/products →
 - VendingMachineController.getAvailableProducts() →
 - VendingMachineService.getAvailableProducts() →
 - Check inventory levels →
 - Return product list with availability
-

2. Payment Processing Use Case:

- POST /api/payment →
- PaymentController.processPayment() →
- PaymentService.processPayment() →
- Validate inventory →
- Check state (must be IdleState) →
- Change state to ProcessingPaymentState →
- Process payment →
- Update inventory →
- Change state to DispensingState →
- Dispense product →

- Change state back to IdleState →
 - Return transaction
-

3. Payment Cancel/Failure Use Case:

- POST /api/payment/cancel →
 - PaymentController.cancelPayment() →
 - PaymentService.cancelPayment() →
 - Rollback inventory →
 - Refund money →
 - Log failure →
 - Change state back to IdleState →
 - Reset state
-

4. Admin Stocking Use Case:

- POST /api/admin/stock →
 - AdminController.restockProduct() →
 - AdminService.restockProduct() →
 - Validate admin →
 - Check Current state →
 - Update inventory →
 - Reset alerts →
 - Log activity →
 - Ensure state is IdleState
-

5. Power Failure Recovery Use Case:

- System startup →
 - `RecoveryController.checkAndRecover()` →
 - `RecoveryService.performRecovery()` →
 - Check pending recoveries →
 - Check last known state →
 - Execute recovery based on interrupted state →
 - Reset to safe state (`IdleState`) →
 - Mark complete
-

Step 6: Apply OOP Principles & Design Patterns

The system design leverages key design patterns and OOP principles to ensure robust state handling, modular structure, and easy scalability across intersections.

Design Patterns Used:

- **Repository Pattern** – for data access abstraction
 - **Service Layer Pattern** – for business logic separation
 - **State Pattern** – for vending machine state management
 - **RESTful API Design** – for clean HTTP endpoints
 - **Controller Separation Pattern** – for better separation of concerns
-

OOP Principles Applied:

- **Single Responsibility** – each controller has one clear purpose
- **Open/Closed** – easy to extend with new product types
- **Encapsulation** – domain objects encapsulate their data and behavior

- **Dependency Inversion** – services depend on repositories, not concrete implementations
-

Step 7: Handle Edge Cases

Edge Case Solutions:

- **Insufficient change in machine** – Check available denominations before accepting payment, display error if insufficient.
 - **Product out of stock during transaction** – Check inventory before dispensing, refund payment if out of stock.
 - **Concurrent access to same product** – Use state pattern to ensure thread safety, queue transactions.
 - **Payment failure** – Rollback inventory changes, refund money, log failure reason.
 - **Insufficient funds** – Display current total vs required amount, allow additional payment or cancellation.
-

Implementation Strategies:

- **Concurrent Access:** State pattern ensures thread safety and proper transaction queuing.
- **Change Calculation:** Greedy algorithm to calculate optimal change combination.
- **Inventory Management:** Real-time inventory updates with threshold-based restocking alerts.
- **Transaction Recovery:** Transaction logging for recovery after system failures.
- **Error Handling:** Comprehensive error handling with user-friendly messages.

Step 9: Future Enhancements

In the current design, the following future functional requirements can be introduced:

1. Multiple Vending Machines

- Centralized inventory management
- Remote monitoring and control
- Sales analytics across machines

2. Advanced Payment Methods

- Credit/debit card support
- Mobile payment integration
- Digital wallet support

3. Enhanced Inventory Management

- Expiry date tracking
- Temperature monitoring for perishables
- Automatic reordering

4. Advanced Analytics

- Sales trend analysis

- Popular product identification
 - Revenue Optimization
-

Summary

This simplified design focuses on 4 core API flows perfect for a 60-minute interview:

- **Product Listing:** GET /api/products (inventory management)
 - **Payment Processing:** POST /api/payment (core business logic)
 - **Payment Cancel/Failure:** POST /api/payment/cancel (error handling)
 - **Admin Stocking:** POST /api/admin/stock (maintenance operations)
-

Key Improvements:

- Separated PaymentController for better modularity and Single Responsibility Principle
 - Added RecoveryController to handle power failures and system recovery automatically
-

Key Features Implemented:

- RESTful API design with clear endpoints
- Separate controllers for different concerns (Products, Payments, Recovery, Admin)
- State pattern for machine state management
- Inventory management and restocking

- Payment processing with change calculation
 - Error handling and transaction rollback
 - Simple power failure recovery with Recovery entity
 - Clean recovery process on system startup
 - Admin operations for maintenance
 - Clean architecture with proper separation of concerns
-

Recovery System Highlights:

- **Simple Recovery entity** - Just tracks pending recoveries
- **Automatic recovery** on system startup
- **Clean decision logic** - `PROCESSING_PAYMENT` = refund, `DISPENSING` = dispense + change
- **No complex logging** - Just what's needed for recovery
- **Easy to understand** - Perfect for interview discussion